

1. 일반 함수와 람다식의 차이

① 일반 함수

일반 함수는 특정 작업을 수행하도록 이름을 붙여 정의한 코드 블록이다. 함수는 프로그램의 여러 위치에서 호출하여 재사용할 수 있으며, 일반적으로 반환형, 함수 이름, 매개변수, 함수 본문으로 구성된다.

② 람다식(Lambda Expression)

람다식은 함수의 이름을 별도로 정의하지 않고 필요한 위치에서 직접 작성할 수 있는 익명 함수 객체를 만드는 표현식이다. C++11부터 지원되며, 간단한 기능을 짧은 코드로 구현할 때 유용하다.

람다식의 기본 형식은 `[캡처](매개변수) { 함수 본문 }`이다.

1-2. 일반 함수와 람다식의 코드 비교

① 일반 함수 예제

```
```cpp
#include <iostream>
using namespace std;

int add(int a, int b) {
 return a + b;
}

int main() {
 cout << add(3, 5) << endl;
 return 0;
}
```
```

실행 결과: 8

일반 함수는 `add()`라는 이름으로 함수를 정의하고, `main()` 함수에서 호출하여 사용한다.

② 람다식 예제

```

```cpp
#include <iostream>
using namespace std;

int main() {
 auto add = [](int a, int b) {
 return a + b;
 };

 cout << add(3, 5) << endl;
 return 0;
}
```

```

실행 결과: 8

람다식은 별도의 일반 함수를 정의하지 않고 `main()` 함수 내부에서 필요한 기능을 직접 작성한다.

1-3. 일반 함수와 람다식의 주요 차이점

| 구분 | 일반 함수 | 람다식 |
|----------|-------------------------|------------------------|
| 함수 이름 | 일반적으로 이름이 있음 | 익명 함수 객체 생성 |
| 정의 위치 | 독립적으로 정의 가능 | 표현식이 허용되는 위치에서 작성 가능 |
| 코드 길이 | 별도 함수 정의 필요 | 간단한 기능을 짧게 작성 가능 |
| 외부 변수 사용 | 매개변수 전달 또는 접근 가능한 변수 사용 | 캡처 기능으로 주변 지역 변수 사용 가능 |
| 재사용성 | 여러 위치에서 재사용하기 편리 | 주로 특정 위치에서 간단한 작업에 사용 |
| 주요 용도 | 반복적으로 사용하는 기능 | 일회성 작업, 콜백 함수, 알고리즘 |

1-4. 일반 함수 대신 람다식을 사용할 때의 장점

① 코드의 간결성

별도의 함수를 정의할 필요 없이 필요한 위치에서 직접 기능을 구현할 수 있어 코드가 간결해진다.

② 가독성 향상

간단한 작업을 사용하는 위치에 바로 작성할 수 있으므로 코드의 흐름을 이해하기 쉽다.

③ 외부 변수 캡처 기능

람다식은 캡처 기능을 통해 주변 지역 변수를 값이나 참조 형태로 사용할 수 있다. 따라서 별도의 매개변수 전달 없이도 필요한 데이터를 활용할 수 있다.

④ 일회성 함수 작성에 유리

한 번만 사용하는 간단한 기능을 위해 별도의 일반 함수를 정의할 필요가 없어 편리하다.

⑤ STL 알고리즘과 함께 사용하기 편리

`sort()`, `for\_each()`, `find\_if()` 등의 STL 알고리즘에서 람다식을 사용하면 정렬 조건이나 처리 동작을 간단하게 지정할 수 있다.

1-5. 결론

일반 함수와 람다식은 모두 특정 작업을 수행하는 데 사용되지만, 정의 방식과 활용 목적에 차이가 있다.

일반 함수는 여러 곳에서 반복적으로 사용하는 기능에 적합하고, 람다식은 특정 위치에서 간단한 작업을 수행하거나 외부 변수를 활용해야 할 때 유용하다.

따라서 반복적으로 사용하는 기능은 일반 함수로 구현하고, 간단한 일회성 작업이나 STL 알고리즘의 조건 지정에는 람다식을 사용하는 것이 효과적이다.

2-1 인터넷에서 찾은 예제

참고 자료는 C++ 공식 레퍼런스로 널리 사용되는 [cppreference](http://en.cppreference.com)의 `std::function` 설명. 여기에는 일반 함수와 람다식을 `std::function`에 저장하고 다른 함수에 전달하는 예제가 있음.

[cppreference.com](http://en.cppreference.com)

+1

- `std::function` 참고 자료
- C++ 람다식 참고 자료

2-2 C++ 실습 소스 코드와 실행 결과

```
1 // Welcome
2 #include <iostream>
3 #include <vector>
4 #include <functional>
5 using namespace std;
6
7 // 전달받은 함수를 각 숫자에 적용
8 void process(const vector<int>& vec, function<void(int)> func) {
9     for (int num : vec) {
10         func(num);
11     }
12 }
13
14 // 숫자를 출력하는 일반 함수
15 void print(int num) {
16     cout << num << " ";
17 }
18
19 int main() {
20     vector<int> vec = {1, 2, 3, 4, 5};
21
22     // 일반 함수를 인자로 전달
23     process(vec, print);
24     cout << endl;
25
26     // 람다식을 인자로 전달
27     int sum = 0;
28     process(vec, [&sum](int num) {
29         sum += num;
30     });
31
32     cout << "sum=" << sum << endl;
33     return 0;
34 }
```

1 2 3 4 5
sum=15

2-3. 소스 코드 자세한 설명 (과제 제출용)

3. 소스코드 자세한 설명

실습 목적

C++의 `std::function`과 람다식을 이용하여 함수를 다른 함수의 인자로 전달하는 방법을 이해한다. 일반 함수와 람다식을 각각 인자로 전달하여 실행 결과를 비교한다.

3-2

2. 소스 코드 설명

① 헤더 파일 선언

- `#include <iostream>` : 화면에 데이터를 출력하기 위한 헤더 파일이다.
- `#include <vector>` : 여러 정수를 저장하는 `vector` 컨테이너를 사용하기 위한 헤더 파일

이다.

- `#include <functional>` : `std::function`을 사용하기 위한 헤더 파일이다.
- `using namespace std;` : `std::`를 생략하고 표준 라이브러리 기능을 사용할 수 있게 한다.

② process 함수 정의

```
void process(const vector<int>& vec, function<void(int)> func)
```

정수 벡터와 함수를 인자로 전달받는 함수이다.

- `const vector<int>& vec` : 벡터를 복사하지 않고 참조로 전달받으며, 함수 내부에서 수정하지 못하도록 한다.
- `function<void(int)> func` : 정수 하나를 전달받고 반환값이 없는 함수를 저장하여 호출할 수 있는 객체이다.
- `for (int num : vec)` : 벡터에 저장된 숫자를 하나씩 가져온다.
- `func(num)` : 전달받은 함수를 각 숫자에 대해 실행한다.

③ print 함수 정의

```
void print(int num)
```

전달받은 정수를 화면에 출력하는 일반 함수이다.

`cout << num << " ";`는 숫자와 공백을 출력한다.

④ main 함수와 벡터 생성

```
vector<int> vec = {1, 2, 3, 4, 5};
```

1부터 5까지의 정수를 저장하는 벡터를 생성한다.

⑤ 일반 함수를 인자로 전달

```
process(vec, print);
```

`process` 함수에 벡터와 일반 함수 `print`를 전달한다.

`process` 함수는 벡터의 각 숫자를 `print` 함수에 전달하므로 1 2 3 4 5가 순서대로 출력된다.

⑥ 람다식을 인자로 전달

```

```cpp
int sum = 0;
process(vec, [&sum](int num) {
 sum += num;
});
```

```

- ``int sum = 0;`` : 숫자의 합계를 저장할 변수를 선언하고 0으로 초기화한다.
- ``[&sum]`` : 외부 변수 `sum`을 참조 방식으로 캡처하여 람다식 내부에서 변경할 수 있게 한다.
- ``(int num)`` : `process` 함수에서 전달한 정수를 받는 매개변수이다.
- ``sum += num;`` : 전달받은 숫자를 `sum`에 누적하여 더한다.

따라서 1+2+3+4+5가 계산되어 `sum`에는 15가 저장된다.

⑦ 최종 결과 출력

```

`cout << "sum=" << sum << endl;`

```

계산된 합계를 화면에 출력한다.

실행 결과

```

```text
1 2 3 4 5
sum=15
```

```

실습 결과 및 고찰

이번 실습에서는 `std::function`을 사용하여 일반 함수와 람다식을 동일한 함수의 인자로 전달하였다.

일반 함수 `print`를 전달했을 때는 벡터의 숫자가 출력되었고, 람다식을 전달했을 때는 벡터의 숫자를 모두 더한 결과가 출력되었다.

특히 람다식의 참조 캡처 기능을 통해 외부 변수 `sum`의 값을 직접 변경할 수 있다는 점을 확인하였다.

이를 통해 `std::function`을 사용하면 하나의 함수에서 서로 다른 동작을 유연하게 수행할 수

있으며, 람다식을 이용하면 별도의 함수를 정의하지 않고 필요한 기능을 간결하게 구현할 수 있음을 알 수 있었다.